3장 컴포넌트

3.1 컴포넌트란
3.2 컴포넌트의 종류
3.3 컴포넌트 기초
3.4 컴포넌트와 props 객체
3.5 컴포넌트와 이벤트

3.1 컴포넌트란

- 컴포넌트 - 객체지향 언어 용어로 화면 UI를 담당하는 클래스
- 클래스 컴포넌트와 함수 컴포넌트 두 가지
- 컴포넌트는 리액트가 제공하는 기본 컴포넌트와 리액트 개발자가 필요에 의해 만드는 사용자 컴포넌트 두 가지가 존재
- 리액트 제공 기본 컴포넌트는 소문자로 시작(예:)
- 사용자 컴포넌트는 대문자로 시작하는 낙타등 표기법 사용(예: <HelloWorld/>)

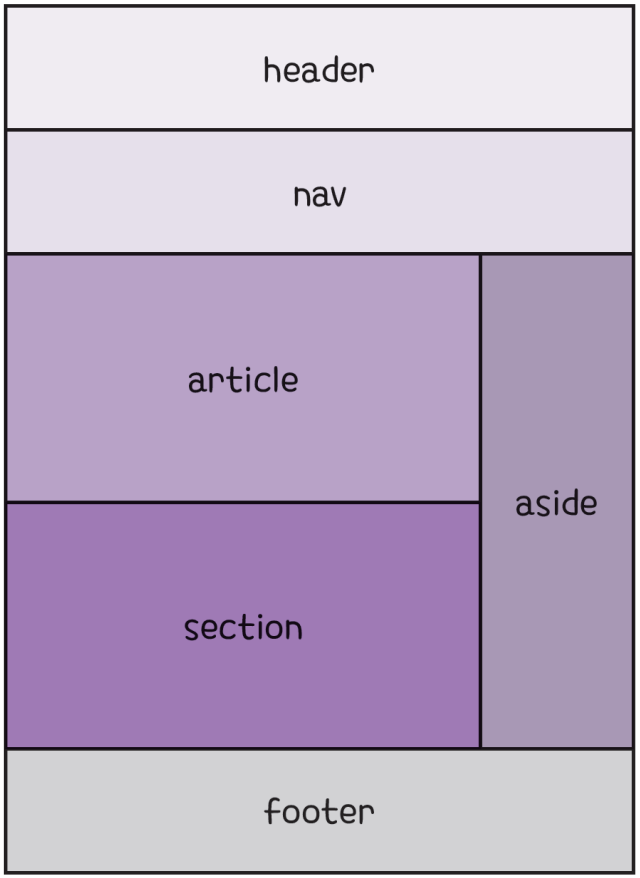
3.1 컴포넌트란

- 컴포넌트(component): UI를 구성하는 독립적이며 재사용 가능한 작은 단위
- React 는 이러한 컴포넌트를 조합해 하나의 애플리케이션을 완성함.

3.1 컴포넌트란

- 추상화(abstraction): 웹 페이지를 기능 단위로 나누는 과정
- 추상화는 복잡한 구조를 단순하게 표현함으로써 전체 시스템을 더 명확하게 이해하고 관리할 수 있도록 해 줌

▶ 웹 페이지 추상화



```
1   //Header 컴포넌트 : 로고와 내비게이션 바
2   function Header() {
3     return (
4       <header>
5         <h1>My Website</h1>
6         <Nav />
7       </header>
8     );
9   }
10  //Navigation 컴포넌트 : 웹사이트 메뉴
11  function Nav() {
12    return <nav>Home | About | Services | Contact</nav>;
13  }
14  //Article 컴포넌트 : 주요 콘텐츠
15  function Article() {
16    return (
17      <article>
18        <h2>Main Article</h2>
19        <p>Welcome to the main content area
20          of this website.</p>
21        <Section />
22      </article>
23    );
24  }
```

```
25  //Section 컴포넌트 : 세부 콘텐츠
26  function Section() {
27    return (
28      <section>
29        <h3>More Details</h3>
30        <p>This section provides additional information.</p>
31      </section>
32    );
33  }
34  //Aside 컴포넌트 : 광고, 추가 정보
35  function Aside() {
36    return (
37      <aside>
38        <h3>Related Links</h3>
39        <ul>
40          <li>React Guide</li>
41          <li>Vite Documentation</li>
42          <li>Component Examples</li>
43        </ul>
44      </aside>
45    );
46  }
```

```
//Footer 컴포넌트 : 저작권 정보, 연락처
function Footer() {
  return (
    <footer>
      <p>© 2025 My Website. All rights reserved.</p>
    </footer>
  );
}
//App 컴포넌트 : 모든 컴포넌트를 조합해 전체 페이지 구성
export default function App() {
  return (
    <div>
      <Header />
      <Article />
      <Aside />
      <Footer />
    </div>
  );
}
```

3.1 컴포넌트란

- 컴포넌트 단위로 구성할 때의 이점
  1. 재사용 가능
  2. 유지보수 용이
  3. 로직 분리 가능
  4. 복잡한 상태 관리 가능

3.1 컴포넌트란

- 컴포넌트 단위로 구성할 때의 이점

1. 재사용 가능

- 컴포넌트의 가장 큰 장점 중 하나는 한 번 만든 UI와 기능을 여러 곳에서 다시 사용할 수 있다는 점

✓예를 들어 버튼이 여러 화면에서 반복해서 필요하다고 가정

- 매번 HTML과 스타일을 새로 작성하는 대신, Button 컴포넌트를 한 번 만들어 두면 여러 곳에서 재사용 가능

3.1 컴포넌트란

- 컴포넌트 단위로 구성할 때의 이점

1. 재사용 가능

```
function Button({ text }) {
  return <button>{text}</button>
}
```

```
<Button text="저장" />
<Button text="삭제" />
<Button text="수정" />
```

이 방식의 장점

- 같은 형태의 UI를 반복 작성하지 않아도 됨
- 디자인 통일이 쉬움 / 수정이 필요할 때 한 곳만 고치면 됨

✓ 예를 들어 버튼 스타일을 바꾸고 싶다면 Button 컴포넌트만 수정하면 됨. 프로젝트 전체에서 사용하는 버튼이 모두 일관되게 바뀜

3.1 컴포넌트란

- 컴포넌트 단위로 구성할 때의 이점

  2. 유지보수 용이
     - React에서 컴포넌트 단위 설계는 유지보수성을 크게 높임
     - 기존 기능 수정
     - 오류 수정
     - 디자인 변경
     - 새로운 기능 추가

3.1 컴포넌트란

- 컴포넌트 단위로 구성할 때의 이점

3. 로직 분리 가능
   - 컴포넌트 단위 설계는 화면만 분리하는 것이 아니라 **기능과 로직도 역할별로 분리**
   - **장점**
     - 코드의 책임이 명확해짐
     - 테스트하기 쉬움
     - 같은 로직을 다른 컴포넌트에서도 재사용 가능
     - 디버깅이 쉬워짐

     ✓React에서는 이런 흐름이 특히 중요
     왜냐하면 React는 UI를 컴포넌트로 나누는 구조이면서 동시에 상태, 이벤트, 렌더링 로직을 함께 다루는 방식이기 때문

3.1 컴포넌트란

- 컴포넌트 단위로 구성할 때의 이점

4. 복잡한 상태 관리 가능

- React에서 상태(state)는 매우 중요한 개념. 상태란 화면에 영향을 주는 데이터.
- 예:
  - 로그인 여부 / 장바구니 목록 / 입력값 / 체크박스 선택 여부
  - 모달 열림/닫힘 , 서버에서 받아온 데이터

✓작은 프로젝트에서는 상태 관리가 단순할 수 있지만, 화면이 커지고 기능이 많아지면 상태가 매우 복잡해 짐

✓컴포넌트 단위 구조는 단순한 분할이 아니라 복잡한 상태를 작은 책임 단위로 나누고, 필요한 범위에서 안전하게 관리할 수 있게 해주는 기반 구조

3.1 컴포넌트란

- 컴포넌트 나누기: 목적에 맞게 적절한 수준으로 나눌 것
  - 너무 세분하면 관리가 어렵고
  - 너무 뭉뚱그리면 재사용성과 유지보수성이 떨어진다

3.2 컴포넌트의 종류

3.2.1 클래스 컴포넌트

- 클래스 컴포넌트(class component): ES6에서 도입된 클래스 문법을 사용해 컴포넌트 정의

형식

```
import { Component } from 'react';

class [컴포넌트_이름] extends Component {
  // JSX를 반환하는 render() 메서드
  render() {
    return (...)
  }
}
export default [컴포넌트_이름];
```

3.1 컴포넌트란

- ES6 == ECMAScript 2015
  - ES6 은 React에서 많이 사용하는 현대 JavaScript 문법 (ES6) 기반 코드 스타일
- ECMAScript 는 JavaScript의 표준 규격(specification)
- JavaScript 언어의 문법, 기능, 동작 방식을 정의한 표준
- 관리 기관: Ecma International
- JavaScript 엔진들은 이 표준을 기반으로 구현됨 (예: V8 JavaScript engine)
- ECMA = European Computer Manufacturers Association : IT 및 컴퓨팅 기술 표준을 만드는 국제 표준화 기구

3.2 컴포넌트의 종류

3.2.2 함수형 컴포넌트

- 함수형 컴포넌트(functional component): 함수를 사용해 컴포넌트를 작성하는 방식

형식

```
export default function [컴포넌트_이름]() {
  return (...); // JSX 반환
}
```

- 함수형 컴포넌트의 이름은 파스칼 케이스(PascalCase)로 작성
  - 파스칼 케이스(PascalCase)
    - 각 단어의 첫글자를 대문자로 쓰고 공백이나 구분자 없이 이어 붙이는 표기법

3.2 컴포넌트의 종류

3.2.3 클래스 컴포넌트와 함수형 컴포넌트 사용

- 클래스 컴포넌트는 문법이 복잡하고, 상태 관리나 코드 재사용이 어렵다는 단점
- 함수형 컴포넌트는 단순히 function 키워드로 컴포넌트를 정의할 수 있어서 자바스크립트를 익힌 사람에게 더 직관적
- 리액트 훅의 도입으로 상태 관리, 생명주기 처리 등 대부분의 기능을 함수형 컴포넌트에서 구현할 수 있게 됨
- 리액트 16.8이 발표된 2019년 이후 대부분 프로젝트에서는 함수형 컴포넌트를 주로 사용

3.3 컴포넌트 기초

3.3.1 컴포넌트 확장자

- 확장자에 따라 JSX 문법을 사용할 수 있는지 여부가 달라짐

▶ CRA와 Vite 기반 프로젝트의 확장자별 JSX 지원 여부

확장자	CRA	Vite
.js	JSX 지원	JSX 지원
.jsx	JSX 지원	JSX 지원
.ts	JSX 지원 안 함	JSX 지원 안 함
.tsx	JSX 지원	JSX 지원

3.3 컴포넌트 기초

3.3.2 컴포넌트 정의

●함수형 컴포넌트 정의

- 함수 선언문 방식: 컴포넌트는 function 키워드로 정의하고 JSX를 return 문 안에 작성. 이 컴포넌트를 다른 파일에서 사용할 수 있도록 내보내기(export) 함.

```
export default function App()
{
  return (
    <>
      <h1>Hello, Function Component!</h1>
    </>
  );
}
```

3.3 컴포넌트 기초

3.3.2 컴포넌트 정의

●함수형 컴포넌트 정의

- 컴포넌트 내보내기 방식에 따른 불러오기 차이
  - export 방식: 불러오는 쪽에서 중괄호({})를 사용해 불러와야 함
  - export default 방식: 중괄호 없이 바로 불러올 수 있음(초보자 권장)

```
import { App } from './App.tsx'; // export function App() { ... } 사용 시
import  App  from './App.tsx'; // export default function App() { ... } 사용 시
```

3.3 컴포넌트 기초

3.3.2 컴포넌트 정의

- 함수형 컴포넌트 정의
  - 다른 정의 방식
    - 함수 표현식
    - 화살표 함수(arrow function)

3.3 컴포넌트 기초

- 함수형 컴포넌트 정의
  - 다른 정의 방식 - 함수 표현식

함수 표현식

```
const App = function App() {
  return (
    <>
      <h1>Hello, Function Component!</h1>
    </>
  );
};
export default App;
```

3.3 컴포넌트 기초

- 함수형 컴포넌트 정의
  - 다른 정의 방식 – 화살표 함수

화살표 함수

```
const App = () => {
  return (
    <>
      <h1>Hello, Function Component!</h1>
    </>
  );
};
export default App;
```

3.3 컴포넌트 기초

3.3.3 컴포넌트 추가

- React 애플리케이션은 여러 컴포넌트 파일이 결합되어 하나의 화면을 구성하므로 화면에 여러 컴포넌트를 함께 렌더링하는 방법을 익히는 것이 중요함
- 웹브라우저 화면에 표시되는 시작점은 App 컴포넌트
- 여기에 새로운 컴포넌트((예)Header 컴포넌트)를 하나 만들어 함께 표시

3.3.3 컴포넌트 추가

- 여기에 새로운 컴포넌트((예)Header 컴포넌트)를 하나 만들어 함께 표시

```
function Header() {  ------------ ❶
  return (
    <>
      <h1>Header!</h1>
    </>
  );
}
export default function App() {
  return (
    <>
      <Header />  ---------------- ❷
      <h1>Hello, Function Component!</h1>
    </>
  );
}
```

3.3.3 컴포넌트 추가

- 같은 방식으로 Main과 Footer 컴포넌트도 추가

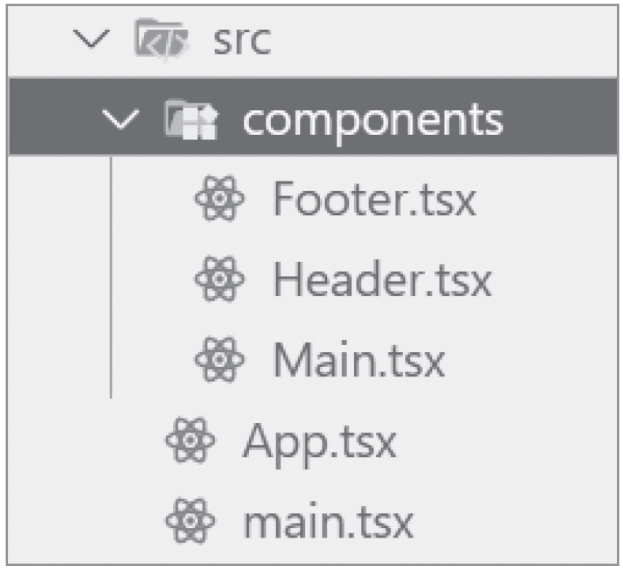
```
function Header() {
  return (
    <>
      <h1>Header!</h1>
    </>
  );
}
function Main() {
  return (
    <>
      <h1>Hello, Function Component 추가 연습!</h1>
    </>
  );
}
function Footer() {
  return (
    <>
      <h1>Footer!</h1>
    </>
  );
}
export default function App()
{
  return (
    <>
      <Header />
      <Main />
      <Footer />
    </>
  );
}
```

3.3 컴포넌트 기초

●컴포넌트 파일 분리

- App.tsx 파일 안에 만든 Header, Main, Footer 컴포넌트는 파일 단위로 분리

▶ 추가한 폴더와 파일 구조



- 다른 파일에 있는 컴포넌트를 사용할 때

형식

```
import 컴포넌트_이름 from '파일_경로';
```

3.3 컴포넌트 기초

●컴포넌트 파일 분리

src/components/Header.tsx

```
export default function Header() {
  return (
    <>
      <h1>Header!</h1>
    </>
  );
}
```

src/components/Main.tsx

```
export default function Main() {
  return (
    <>
      <h1>Hello, Function Component!</h1>
    </>
  );
}
```

src/components/Footer.tsx

```
export default function Footer() {
  return (
    <>
      <h1>Footer!</h1>
    </>
```

3.3 컴포넌트 기초

●컴포넌트 파일 분리

src/App.tsx

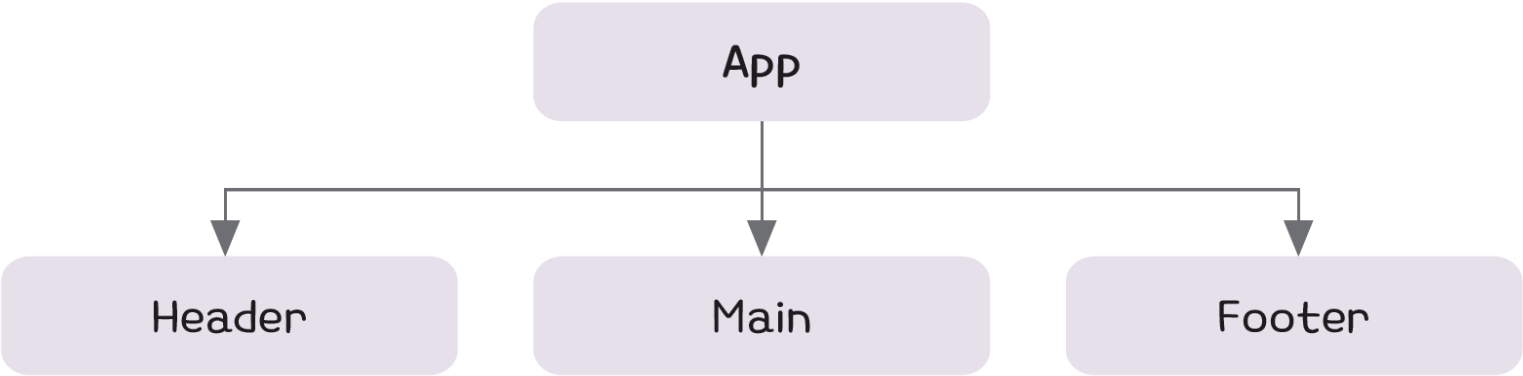
```
import Header from './components/Header';
import Main from './components/Main';
import Footer from './components/Footer';

export default function App() {
  return (
    <>
      <Header />
      <Main />
      <Footer />
    </>
  );
}
```

3.3 컴포넌트 기초

3.3.4 컴포넌트 트리

- 컴포넌트 트리(component tree): 하나의 컴포넌트 안에 또 다른 컴포넌트를 포함하는 방식으로 형성된 계층적 구조
- 루트 컴포넌트(root component): 리액트 애플리케이션에서 가장 바깥쪽에 위치하는 컴포넌트

▶ 컴포넌트 트리



3.3 컴포넌트 기초

- 리액트 컴포넌트의 주요 특징(1)
  - 계층 구조: 컴포넌트는 부모-자식 관계의 계층 구조
    - 부모 컴포넌트는 여러 자식 컴포넌트를 포함 가능.
    - 자식 컴포넌트는 특정 부모 안에서만 렌더링됨
    - HTML에서는 형제 태그가 존재하지만, 리액트의 컴포넌트는 형제 관계라는 개념은 명시적으로 존재하지 않음.
  - 단방향 데이터 흐름
    - 데이터가 항상 부모에서 자식으로만 흐름.
    - 부모 컴포넌트는 props를 통해 자식에게 데이터를 전달할 수 있지만, 자식 컴포넌트는 부모의 데이터를 직접 변경 불가함.

3.3 컴포넌트 기초

- 리액트 컴포넌트의 주요 특징(2)
  - 재사용성: 한 번 정의한 컴포넌트는 여러 위치에서 반복해서 사용할 수 있음. 이를 통해 코드의 중복을 줄이고 유지보수 성능을 향상시킴
  - 상태 관리: 각 컴포넌트는 자신만의 상태(state)를 가짐
  - 렌더링 최적화: 가상 DOM을 사용해 실제 DOM과의 차이를 비교한 뒤 변경된 부분만 최소한으로 업데이트

3.4 컴포넌트와 props 객체

3.4.1 props(properties) 객체란

❖ React 에서는 부모 컴포넌트가 자식 컴포넌트에 데이터를 전달하는 것이 가능하며 이때 사용하는 것이 바로 props(properties).

- props(properties): 컴포넌트를 마치 HTML 태그처럼 사용해 값을 속성 형태로 전달
- (예) App 컴포넌트에서 User 컴포넌트로 “철수 " 라는 데이터를 전달

src/App.tsx

```
import User from './components/User';

export default function App() {
  return <User name='철수' />;
}
```

3.4 컴포넌트와 props 객체

3.4.1 props(properties) 객체란

src/App.tsx

```
import User from './components/User';

export default function App() {
  return <User name='철수' />;
}
```

- 위 코드에서 <User name="철수" />는 User 컴포넌트를 화면에 렌더링하면서 name 이라는 속성의 값으로 "철수 " 를 함께 전달함.

3.4 컴포넌트와 props 객체

3.4.1 props(properties) 객체란

- 여러 데이터를 전달하고 싶다면 속성을 여러 개 추가하면 됨
- (예) name='철수 ' 와 age={20} 이라는 두가지 속성을 User 컴포넌트로 전달함.
  - 이때 문자열은 따옴표(" 또는 ')로, 숫자나 표현식은 중괄호({ })로 감싸서 전달함.

src/App.tsx

```
import User from "./components/User";
export default function App() {
  return <User name='철수' age={20} />;
}
```

3.4 컴포넌트와 props 객체

3.4.1 props(properties) 객체란

- React 에서는 부모 컴포넌트가 자식 컴포넌트에 전달한 모든 속성을 하나의 객체로 모아서 자식 컴포넌트로 전달함
- 부모 컴포넌트에서 User 컴포넌트에 name과 age를 전달하더라도 React 내부에서는 이 데이터를 다음과 같이 단일 객체 형태로 전달하는데 이 객체를 props 객체라고 함
  - 즉, props는 부모 컴포넌트가 자식 컴포넌트에게 넘겨주는 모든 데이터가 담긴 객체임

```
{
    name : '철수' ,
    age : 20
}
```

3.4 컴포넌트와 props 객체

3.4.1 props(properties) 객체란

- 자식 컴포넌트는 전달받은 props 객체를 함수의 매개변수로 받음.
- 매개변수 이름은 props를 사용하지만, 꼭 정해진 것은 아니며 다른 이름을 사용하는 것도 가능함.
- User 컴포넌트에 어떤 값이 전달되는지 확인해 봄
  - src/components 폴더에 User.tsx 파일을 생성하고 다음과 같이 코드를 작성함.

src/components/User.tsx

```
export default function User(props) {  ------- ❶
  console.log(props);  ------------------------- ❷
  return <div>User Component</div>;  --------- ❸
}
```

3.4 컴포넌트와 props 객체

3.4.1 props(properties) 객체란

src/components/User.tsx

```
export default function User(props) {  ------- ❶
  console.log(props);  ------------------------- ❷
  return <div>User Component</div>;  --------- ❸
}
```

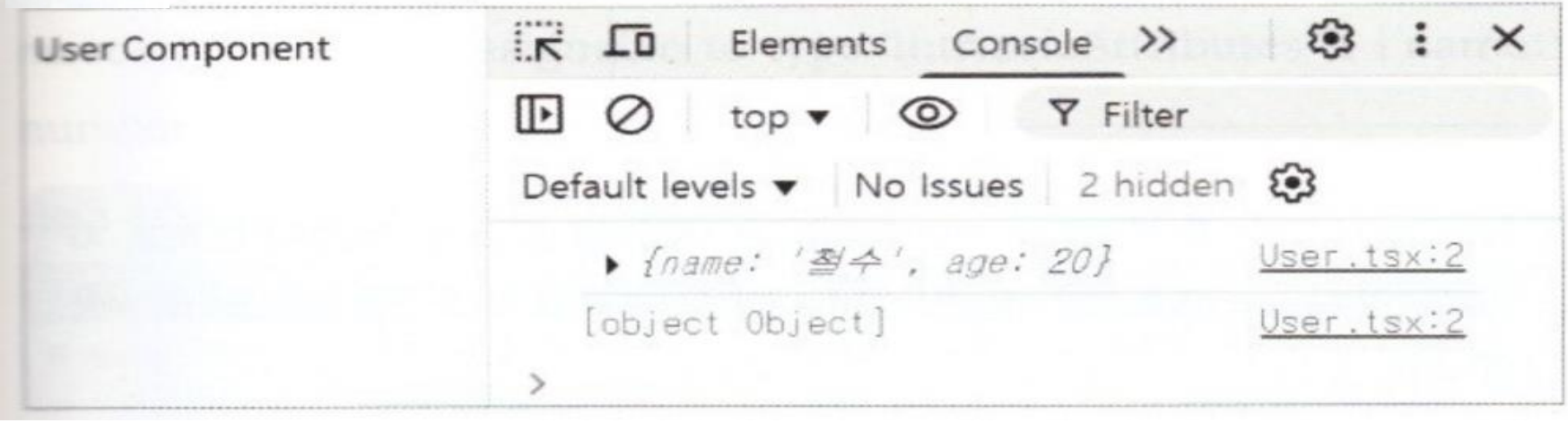
- ① props 는 부모(App) 컴포넌트에서 전달한 모든 속성을 하나의 객체로 받음. 현재 props 밑에 빨간 밑줄이 생겨 오류 표시처럼 보이는데 타입을 지정하지 않아서 생기는 현상이며 일단은 무시함
- ② 전달받은 데이터를 console.log(props)로 콘솔에 출력해 봄
- ③ 화면에 User Component 를 렌더링 함.

3.4 컴포넌트와 props 객체

3.4.1 props(properties) 객체란

- 애플리케이션을 실행하고 웹 브라우저에서 기능키 F12 을 눌러 개발자 도구를 오픈하고 Console 탭을 선택하면 아래와 같이 출력됨
- 데이터는 개별 값이 아닌 하나의 객체로 묶여 props 로 전달된다는 점을 확인 가능.

User 컴포넌트에 전달된 데이터 확인



3.4 컴포넌트와 props 객체

3.4.1 props(properties) 객체란

- 자식 컴포넌트는 props 객체를 통해 부모 컴포넌트가 전달한 데이터를 받아 사용할 수 있음
- props는 객체이기 때문에 자식 컴포넌트에서는 객체 형식으로 접근해야 함

3.4 컴포넌트와 props 객체

3.4.1 props(properties) 객체란

- 자바스크립트에서는 props에 타입을 지정하지 않아도 실행에는 문제가 없음
- 타입스크립트에서는 props(properties) 객체의 구조와 타입을 지정해 주는 것이 원칙이며 좋음
- 타입을 지정하지 않으면 코드 에디터의 자동 완성기능이나 오류 검사가 제대로 동작하지 않을 수 있음

3.4 컴포넌트와 props 객체

3.4.1 props(properties) 객체란

- User 컴포넌트의 타입을 다음과 같이 수정함

src/components/User.tsx

```
export default function User(props: { name: string; age: number }) {  ---------- ❶
  return (
    <>
      <p>name: {props.name}</p>
      <p>age: {props.age}</p>  ---------- ❷
    </>
  );
}
```

① props 의 타입을 명시함. 이 객체는 name(문자열)과 age(숫자) 속성을 가져야 함. 잘못된 타입을 전달하면 타입 스크립트가 컴파일 시점에 오류를 알려줌.

② 속성에는 props.name , props.age 처럼 표기법을 사용해 접근함. props 는 객체이므로 해당 키를 통해 값을 꺼내 사용하는 방식임.

3.4 컴포넌트와 props 객체

3.4.2 props 객체 타입 알아내기

- 앞의 예제에서는 name과 age 를 각각의 속성으로 따로 전달하였고 이처럼 각 데이터를 개별 props 로 지정할 수도 있지만, 전달할 데이터가 많아질수록 props 를 하나하나 지정하면 코드가 길어지고 복잡해질 수도 있음
- 전달할 데이터가 많아질 때, 하나의 객체나 배열로 묶어 한 번에 전달하는 것이 더 깔끔하고 효율적임
- 객체 하나를 props로 전달하면 필요한 데이터를 그 안에서 꺼내 사용할 수 있음

3.4 컴포넌트와 props 객체

3.4.2 props 객체 타입 알아내기

- 앞의 코드를 객체를 사용해 전달하는 방법으로 변경함

```
import User from "./components/User";

export default function App() {
  const userObj = {
    name: "철수",
    age: 20,
  };

  return <User  userObj={userObj}  />;
}
```

3.4 컴포넌트와 props 객체

3.4.2 props 객체 타입 알아내기

- 객체 하나를 props로 전달하면 필요한 데이터를 그 안에서 꺼내 사용할 수 있음.
  - 즉, User 컴포넌트에서는 **props.userObj** 로 해당 객체에 접근해야 함.
  - 앞의 코드를 작성하면 **userObj** 속성에 빨강색 밑줄(오류)이 생기고 이 부분에 마우스 커서를 가져가면 다음 장에 있는 것과 같은 오류 메시지 발생

3.4 컴포넌트와 props 객체

3.4.2 props 객체 타입 알아내기

- 오류 메시지 내용

```
export default function App() {
  //name과 age값을 담은 객체 userObj를 정의함
  const userObj
    name: "철수
    age: 20,
  };

  //{userObj} 
  //User 컴포넌
  return <User userObj={userObj} />;
}
```

'{ userObj: { name: string; age: number; }; }' 형식은 'IntrinsicAttributes & { name: string; age: number; }' 형식에 할당할 수 없습니다.
  'IntrinsicAttributes & { name: string; age: number; }' 형식에 'userObj' 속성이 없습니다. ts(2322)

(property) userObj: {
    name: string;

3.4 컴포넌트와 props 객체

3.4.2 props 객체 타입 알아내기

- 오류 메시지 발생 원인
  - 현재 User 컴포넌트가 {name : string, age : number}타입의 props 를 기대하고 있는데, {userObj : {name:string, age :number} } 형태로 전달하고 있기 때문에 타입이 맞지 않는다는 의미
  - (해결)이 내용을 그대로 User 컴포넌트의 매개변수 타입으로 지정하면 오류가 해결되며 return 문에서 props.userObj 로 데이터에 접근하면 해결됨.

3.4 컴포넌트와 props 객체

3.4.2 props 객체 타입 알아내기

- 객체 데이터를 자식 컴포넌트에 전달했듯이 이벤트 핸들러 함수도 **props**를 통해 자식 컴포넌트에 전달 가능함
- (예) 버튼을 클릭했을 때 어떤 동작을 실행하고 싶다면 부모 컴포넌트에서 클릭 이벤트 핸들러 함수를 정의하고 자식 컴포넌트에 전달하면 됨

3.4.2 props 객체 타입 알아내기

- 이처럼 함수도 **props**를 통해 함수도 자식 컴포넌트에 전달 가능함.
- 전달한 **props**의 타입을 정확히 모를 때는**VScode**의 오류 메시지를 활용하면 매우 유용한데 오류 메시지가 제대로 작동하려면 다음 두 가지 조건을 만족해야 함
- (조건 1) 자식 컴포넌트 파일이 존재해야 함 – 컴포넌트 코드가 아예 없으면 **VSCode** 는 **props**에 대해 검사하지 않으므로 최소한 다음 처럼 비어 있는 컴포넌트라도 정의해야 함

```
export default  function User(){
        return   <> </> ;
}
```

3.4 컴포넌트와 props 객체

3.4.2 props 객체 타입 알아내기

- **(조건 2) props 매개변수가 없는 상태여야 함**
  - **VSCode 는 props 매개변수가 이미 선언되어 있으면 오류 메시지를 표시하지 않음**
  - **오류 메시지로부터 정확한 props 타입을 추론하려면 props 매개변수를 제거한 상태여야 함**

3.4 컴포넌트와 props 객체

3.4.3 props 객체의 구조 분해 할당과 타입 정의

- 지금까지 작성한 User 컴포넌트의 전체 코드

src/components/User.tsx

```
export default function User(props: {
  userObj: { name: string; age: number };
  clickHandler: () => void;
}) {
  return (
    <>
      <p>name: {props.userObj.name}</p>
      <p>age: {props.userObj.age}</p>
      <button onClick={props.clickHandler}>클릭</button>
    </>
  );
}
```

3.4 컴포넌트와 props 객체

3.4.3 props 객체의 구조 분해 할당과 타입 정의

●매개변수에서의 구조 분해 할당

- props는 단일 객체이므로 부모 컴포넌트에서 속성을 몇십 개 지정해도 결국 하나의 props 객체로 전달됨
- 이때 구조 분해 할당을 사용하면 객체의 속성을 더 간단하게 변수에 할당 가능함
- 구조 분해 할당(Destructuring assignment)
  - 객체나 배열의 속성을 추출해 변수에 할당하는 자바스크립트 문법

3.4.3 props 객체의 구조 분해 할당과 타입 정의

●매개변수에서의 구조 분해 할당

- User 컴포넌트를 구조 분해 할당을 사용하도록 바꾸면 다음과 같음
- props 객체는 userObj 속성과 clickHandler 속성을 전달하므로 개별 변수로 분해하면 props . 을 생략하고 더 깔끔하게 사용 가능함

src/components/User.tsx

```
export default function User({ userObj, clickHandler }: { ------ ❶
  userObj: { name: string; age: number; };
  clickHandler: () => void;
}) {
  return (
    <>
      <p>name: {userObj.name}</p> ------------------------------- ❷
      <p>age: {userObj.age}</p>
      <button onClick={clickHandler}>클릭</button>
    </>
  );
}
```

3.4.3 props 객체의 구조 분해 할당과 타입 정의

●매개변수에서의 구조 분해 할당

- { userObj , clickHandler }는 props 객체에서 userObj와 clickHandler 라는 두 속성을 꺼내는 구조 분해 할당 구문
- 구조 분해 할당으로 props를 받아오면 JSX에서 props.userObj.name 대신 userObj.name 으로 간결하게 접근 가능함.

3.4.3 props 객체의 구조 분해 할당과 타입 정의

●매개변수에서의 구조 분해 할당

- userObj 객체의 속성(name, age) 까지 한번에 구조 분해 할당 하면 userObj 도 생략되어 코드가 더욱 간결해 짐.

src/components/User.tsx

```
export default function User({ userObj: { name, age }, clickHandler }: { ------ ❶
  userObj: { name: string; age: number; };
  clickHandler: () => void;
}) {
  return (
    <>
      <p>name: {name}</p> ------------------------------------------------------------ ❷
      <p>age: {age}</p>
      <button onClick={clickHandler}>클릭</button>
    </>
```

3.4.3 props 객체의 구조 분해 할당과 타입 정의

●매개변수에서의 구조 분해 할당

- userObj 객체의 속성(name, age) 까지 한번에 구조 분해 할당하면 userObj 도 생략되어 코드가 더욱 간결해 짐.
  - { userObj: {name, age}...} ->userObj 객체를 다시 구조 분해 할당해서 name과 age 값을 꺼냄.
  - 구조 분해 할당은 매개변수에서 바로 처리되므로 userObj.name, userObj.age 대신 name, age만 사용하면 됨.

3.4.3 props 객체의 구조 분해 할당과 타입 정의

●함수 내부에서의 구조 분해 할당

- 구조 분해 할당을 꼭 매개변수에서 하지 않아도 됨.
  - props객체를 그대로 받아 함수 내부에서 분해해도 같은 효과를 얻을 수 있음

src/components/User.tsx

```
export default function User(props: {
  userObj: { name: string; age: number; };
  clickHandler: () => void;
}) {
  const {
    userObj: { name, age },
    clickHandler,
  } = props;
  return (
    <>
      <p>name: {name}</p>
      <p>age: {age}</p>
      <button onClick={clickHandler}>클릭</button>
    </>
  );
}
```

3.4.3 props 객체의 구조 분해 할당과 타입 정의

●함수 내부에서의 구조 분해 할당

- 구조 분해 할당을 꼭 매개변수에서 하지 않아도 됨.
  - props객체를 그대로 받아 함수 내부에서 분해해도 같은 효과를 얻을 수 있음
    - 이 방식은 props 전체를 일단 받고, 구조 분해 할당을 코드 흐름 중간에서 명확하게 처리하고 싶을 때 유용함
    - 함수 선언부가 짧아서 함수 시그니처를 더 깔끔하게 만들 수 있음

3.4.3 props 객체의 구조 분해 할당과 타입 정의

●타입 정의

- 앞의 예제처럼 userObj와 clickHandler 속성을 가진 props 객체의 타입을 한 줄에 모두 작성하면 함수 매개변수의 타입이 너무 길고 복잡해 보일 수 있음.
  - 이럴 때는 TypeScript의 인터페이스(interface)를 사용해 props 객체의 타입을 따로 정의 가능함.

●타입 정의

src/components/User.tsx

```
interface UserProps { // 인터페이스로 props 타입 분리
  userObj: { name: string; age: number; };
  clickHandler: () => void;
}
```

interface 대신 type 키워드를 사용해도 동일한 효과를 얻을 수 있음.
type 키워드를 사용해 타입을 정의하는 것을 타입 별칭(type aliasing)이라고 함

```
export default function User(props: UserProps) {
  const {
    userObj: { name, age },
    clickHandler,
  } = props;
  return (
    <>
      <p>name: {name}</p>
      <p>age: {age}</p>
      <button onClick={clickHandler}>클릭</button>
    </>
  );
}
```

3.4.3 props 객체의 구조 분해 할당과 타입 정의

●타입 정의

- Interface 로 타입을 분리하니 props 의 타입이 UserProps 로 깔끔하게 정리되어 함수 선언부가 훨씬 보기 쉬워짐.
- 이렇게 정의한 인터페이스를 React에서는 .d.ts 파일로 분리 가능함
  - 컴포넌트 파일이 깔끔해지고, 타입 재사용성도 높아짐
  - 이러한 타입 선언 전용 파일(.d.ts)은 일반적으로 src/types 폴더에 모아 관리하며 이 파일은 따로 import 하지 않아도 타입을 사용 가능함.

3.4.3 props 객체의 구조 분해 할당과 타입 정의

●타입 정의

src/types/props.d.ts

```
interface UserProps {
  userObj: { name: string; age: number; };
  clickHandler: () => void;
}
```

***.d.ts 파일을 별도로 만들어 타입 정의를 하는 방식은 실무에서 자주 활용하므로 꼭 알아두자~!**

3.4.3 props 객체의 구조 분해 할당과 타입 정의

●타입 정의

src/component/User.tsx

```
export default function User(props: UserProps) {
  const {
    userObj: { name, age },
    clickHandler,
  } = props;
  return (
    <>
      <p>name: {name}</p>
      <p>age: {age}</p>
      <button onClick={clickHandler}>클릭</button>
    </>
  );
}
```

3.4.4 props와 전개 연산자

- 전개 연산자(spread operator): userObj 객체의 속성을 각각 전달하고 싶다면, 전개 연산자(...)를 사용

- 현재 App 컴포넌트의 코드는 다음과 같음.(src/App.tsx)

```
import User from "./components/User";
export default function App() {
  const userObj = {
    name: "철수",
    age: 20,
  };
  const clickHandler = () => {
    console.log("clicked");
  };
  return <User userObj={userObj} clickHandler={clickHandler} />;
}
```

3.4.4 props와 전개 연산자

- 만약 userObj 객체의 속성을 각각 전달하고 싶다면, 전개 연산자 (…) 을 사용 가능함
- 전개 연산자(spread operator) ' … '는 객체의 속성을 풀어 해당 위치에 하나하나 나열해 주는 역할을 수행

src/App.tsx

```
import User from './components/User';

export default function App() {
  const userObj = {
    name: '철수',
    age: 20,
  };
  const clickHandler = () => {
    console.log('clicked');
  };
  return <User {...userObj} clickHandler={clickHandler} />;
}
```

3.4.4 props와 전개 연산자

- 이와 같이 작성하면 props에 name, age, clickHandler 라는 3개의 속성이 개별로 전달되는 효과가 있음

```
return <User name='철수' age={20} clickHandler={clickHandler} />;
```

3.4.4 props와 전개 연산자

- 자식 컴포넌트에서는 **userObj** 객체의 속성들이 분리되어 전달되므로 더 이상 **props.userObj** 로 접근할 수 없음.
- 개별 속성을 직접 **props** 로 받도록 다음과 같이 코드 수정 필.

```
interface UserProps {
  name: string;
  age: number;
  clickHandler: () => void;
}
export default function User(props: UserProps) {
  const { name, age, clickHandler } = props;
  return (
    <>
      <p>name: {name}</p>
      <p>age: {age}</p>
      <button onClick={clickHandler}>클릭</button>
    </>
  );
}
```

✓ 전개 연산자를 사용하면 코드가 간결해지고 필요한 속성만 선택적으로 전달할 수 있어 편리하며 실무에서도 자주 사용함.

3.4 컴포넌트와 props 객체

3.4.5 children 속성

- React 에서 컴포넌트는 기본적으로 자체 종료 태그 형태인 <컴포넌트 /> 로 사용함
- 컴포넌트를 시작 태그와 종료 태그를 함께 사용하는 형태(<컴포넌트>내용</컴포넌트>)로도 사용할 수 있음
  - 이러한 형태를 사용하면 컴포넌트 안에 포함된 내용을 특별한 **props** 속성으로 취급하며 리액트에서는 이를 **children** 이라고 함

3.4 컴포넌트와 props 객체

3.4.5 children 속성

- (예) User 컴포넌트를 시작 태그와 종료 태그 사이에 내용을 포함하는 방식으로 사용 가능함.

src/App.tsx

```
import User from './components/User';

export default function App() {
  return (
    <>
      <User>
        <p>James</p>
        <p>20</p>
        <p>male</p>
      </User>
    </>
  );
}
```

3.4 컴포넌트와 props 객체

3.4.5 children 속성

- 코드에서 < User> 컴포넌트 내부에 작성한 <p>요소 3개는 모두 User 컴포넌트에 children 이라는 속성으로 전달됨
- React 에서는 children 이 텍스트, JSX 요소, 숫자, 배열 등 다양한 형태가 될 수 있기 때문에 이를 유연하게 처리할 수 있는 React.ReactNode 타입으로 지정함

src/components/User.tsx

```
export default function User(props: { children: React.ReactNode }) {
  return <>{props.children}</>;
}
```

3.4 컴포넌트와 props 객체

3.4.5 children 속성

- 이처럼 **props.children** 으로 전달받은 콘텐츠를 출력 가능하며 이때 구조 분해 할당을 사용하면 코드가 더 간결해짐.

src/components/User.tsx

```
export default function User({ children }: { children: React.ReactNode }) {
  return <>{children}</>;
}
```

3.4 컴포넌트와 props 객체

3.4.5 children 속성

- 애플리케이션을 실행하면 <User> 컴포넌트 안에 작성한 <p> 요소들이 화면에 렌더링 되는 것을 확인 가능함.
- 앞으로는 children을 사용할 때 구조 분해 할당 방식을 기본으로 사용할 예정.
- [children 속성으로 받은 <p> 요소를 렌더링한 결과]

3.4 컴포넌트와 props 객체

3.4.5 children 속성

- **children 속성은 props 객체의 일부이므로 다른 속성과 함께 사용 가능**
- **(예) User 컴포넌트에 title 이라는 속성을 추가로 전달해 봄.**

src/App.tsx

```
import User from "./components/User";
export default function App() {
  return (
    <>
      <User title="User Component">
        <p>james</p>
        <p>20</p>
        <p>male</p>
      </User>
    </>
  );
}
```

3.4 컴포넌트와 props 객체

3.4.5 children 속성

- User 컴포넌트에 title 이라는 일반 문자열 속성과 <p> 요소 3개로 구성된 children을 함께 전달 받음
- User 컴포넌트에서는 두 값을 모두 받을 수 있도록 props 타입을 다음과 같이 title 속성과 children 속성을 함께 props 로 받아 처리하면 됨

3.4 컴포넌트와 props 객체

3.4.5 children 속성

src/components/User.tsx

```
export default function User({
  title,
  children,
}: {
  title: string;
  children: React.ReactNode;
}) {
  return (
    <>
      <h1>{title}</h1>
      {children}
    </>
  );
}
```

3.4 컴포넌트와 props 객체

3.4.5 children 속성

- children 속성은 컴포넌트 태그 사이에 작성된 모든 내용을 하나로 묶어 전달하는 특별한 속성임
- 내용이 여러 요소로 구성되었어도 React는 이를 자동으로 하나의 children 값으로 처리하므로 하나의 컴포넌트에 children을 2개 이상 따로 지정하는 것은 불가능함.

3.4 컴포넌트와 props 객체

Note **ref와 key 속성은 props로 전달 불가능**

리액트에서는 보통 부모 컴포넌트가 자식 컴포넌트에 데이터를 전달할 때 props를 사용합니다. 하지만 ref(리액트 18까지)와 key는 예외입니다. 두 속성은 자식 컴포넌트에서 props.ref 또는 props.key로 접근할 수 없습니다. 두 속성은 리액트가 내부적으로 따로 관리하고 props 객체에는 포함하지 않습니다. 이는 ref와 key가 일반적인 데이터 전달용이 아니라 리액트 내부에서 특별한 용도로 사용하는 속성이기 때문입니다.

- **ref**: DOM 요소나 컴포넌트를 직접 참조할 때 사용합니다.
- **key**: 리스트를 렌더링할 때 가상 DOM에서 항목을 효율적으로 식별하고 업데이트하기 위해 사용합니다.

key를 자식 컴포넌트에서 고유 값으로 사용하고 싶다면 id와 같은 별도의 속성으로 전달해야 합니다. ref는 forwardRef를 사용하면 자식 컴포넌트에서 별도의 매개변수로 받을 수 있습니다.

참고로, 리액트 19부터는 ref도 일반 props처럼 전달할 수 있도록 변경되었습니다.